

Find Available Parking Spots

https://gemini.google.com/app/40535060bd85d8ba

User prompt: CREATE TABLE ParkingReservation (id INTEGER PRIMARY KEY AUTOINCREMENT, from_time DATETIME NOT NULL, to_time DATETIME NOT NULL, spot_id INTEGER NOT NULL, user_id INTEGER NOT NULL, car_id INTEGER, FOREIGN KEY (spot_id) REFERENCES ParkingSpot(id), FOREIGN KEY (user_id) REFERENCES User(id), FOREIGN KEY (car_id) REFERENCES Car(id)); I have this table using Microsoft.AspNetCore.Mvc; using ParkingAPI.Data; namespace ParkingAPI.Controllers { [Route("api/[controller]")] [ApiController] public class ReservationsController : ControllerBase { private readonly ParkingDbContext _context; public ReservationsController(ParkingDbContext context) { _context = context; } } } I want it if i input datetime from to i want to retrive all the available spots

Response: To find all available parking spots for a specific time window, you need to check for spots that **do not have any overlapping reservations**.

Two time periods overlap if the reservation's start time is *before* the requested end time, AND the reservation's end time is *after* the requested start time.

Here is how you can implement this endpoint in your ReservationsController using Entity Framework Core:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ParkingAPI.Data;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace ParkingAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ReservationsController : ControllerBase
    {
        private readonly ParkingDbContext _context;

        public ReservationsController(ParkingDbContext context)
        {
            _context = context;
        }

        [HttpGet("available-spots")]
        public async Task<IActionResult> GetAvailableSpots([FromQuery] DateTime fromTime, [FromQuery] DateTime toTime)
        {
            // 1. Validate the input dates
            if (fromTime >= toTime)
            {
                return BadRequest("The 'from' time must be earlier than the 'to' time.");
            }

            // 2. Query the database for available spots
            // A spot is available if it does NOT have any overlapping reservations
            var availableSpots = await _context.ParkingSpots
                .Where(spot => !_context.ParkingReservations.Any(res =>
                    res.spot_id == spot.id &&
                    res.from_time < toTime &&
                    res.to_time > fromTime))
                .ToListAsync();

            return Ok(availableSpots);
        }
    }
}
```

How the logic works:

- [FromQuery] tells the API to read the fromTime and toTime from the URL query string (e.g., GET /api/reservations/available-spots?fromTime=2024-10-25T10:00:00&toTime=2024-10-25T14:00:00).
- _context.ParkingSpots queries your ParkingSpot table (assuming you have this DbSet configured in your ParkingDbContext).
- The .Any(...) method checks for conflicts. The condition res.from_time < toTime && res.to_time > fromTime is the standard mathematical way to check if two time intervals overlap.
- The ! operator reverses it, returning only the spots where absolutely **zero** conflicts exist.

User prompt: private async Task<List<SpotCountDTO>> GetSpotCounts() { return await _context.ParkingSpots .GroupBy(p => p.Type.Type) .Select(g => new SpotCountDTO { Type = g.Key, Count = g.Count() }) .ToListAsync(); } In this format

Response: Here is how you can update the endpoint to return the count of available spots grouped by their type, matching the SpotCountDTO format you provided:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ParkingAPI.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace ParkingAPI.Controllers
{
```

```

[Route("api/[controller]")]
[ApiController]
public class ReservationsController : ControllerBase
{
    private readonly ParkingDbContext _context;

    public ReservationsController(ParkingDbContext context)
    {
        _context = context;
    }

    [HttpGet("available-spots-count")]
    public async Task<ActionResult<List<SpotCountDTO>>> GetAvailableSpotCounts([FromQuery] DateTime fromTime, [FromQuery] DateTime toTime)
    {
        if (fromTime >= toTime)
        {
            return BadRequest("The 'from' time must be earlier than the 'to' time.");
        }

        // 1. Filter for spots with NO overlapping reservations
        // 2. Group the remaining (available) spots by Type
        // 3. Select into your SpotCountDTO format
        var availableSpotCounts = await _context.ParkingSpots
            .Where(spot => !_context.ParkingReservations.Any(res =>
                res.spot_id == spot.id &&
                res.from_time < toTime &&
                res.to_time > fromTime))
            .GroupBy(p => p.Type.Type)
            .Select(g => new SpotCountDTO
            {
                Type = g.Key,
                Count = g.Count()
            })
            .ToListAsync();

        return Ok(availableSpotCounts);
    }
}

```

What changed:

1. **.Where(...)** **comes first**: It filters out any spots that are currently booked during the requested time window before any counting happens.
2. **.GroupBy(...)** **and** **.Select(...)** **chain**: This applies the exact grouping logic from your `GetSpotCounts()` method, but it only groups the spots that survived the `Where` filter.
3. **Return type**: Changed the method signature to strongly type the return as `ActionResult<List<SpotCountDTO>>` for better Swagger documentation and type safety.

User prompt: Make the reservation: input: from to, spot type(string) It should get the first spot with that type that is available

Response: To achieve this, we need a POST endpoint that accepts the reservation details, finds the first available spot of the requested type, and saves the new reservation to the database.

First, create a DTO (Data Transfer Object) to handle the incoming JSON request data. You will need the user's ID (and optionally the car ID) to satisfy the foreign keys in your `ParkingReservation` table.

```

public class CreateReservationDTO
{
    public DateTime FromTime { get; set; }
    public DateTime ToTime { get; set; }
    public string SpotType { get; set; }
    public int UserId { get; set; }
    public int? CarId { get; set; }
}

```

Now, add the following POST endpoint to your `ReservationsController`:

```

[HttpPost("reserve")]
public async Task<ActionResult> MakeReservation([FromBody] CreateReservationDTO request)
{
    // 1. Validate time
    if (request.FromTime >= request.ToTime)
    {
        return BadRequest("The 'from' time must be earlier than the 'to' time.");
    }

    // 2. Find the FIRST available spot that matches the requested type
    var availableSpot = await _context.ParkingSpots
        .Where(spot => spot.Type.Type == request.SpotType) // Filter by the requested spot type
        .Where(spot => !_context.ParkingReservations.Any(res =>
            res.spot_id == spot.id &&
            res.from_time < request.ToTime &&
            res.to_time > request.FromTime)) // Ensure no overlapping reservations
        .FirstOrDefaultAsync(); // Get the first one found, or null if none exist
}

```

```
// 3. If no spot is found, return an error
if (availableSpot == null)
{
    return NotFound($"No available spots of type '{request.SpotType}' for the selected time window.");
}

// 4. Create the new reservation
var newReservation = new ParkingReservation
{
    from_time = request.FromTime,
    to_time = request.ToTime,
    spot_id = availableSpot.id,
    user_id = request.UserId,
    car_id = request.CarId
};

// 5. Save to database
_context.ParkingReservations.Add(newReservation);
await _context.SaveChangesAsync();

// 6. Return success (returning the created reservation)
return Ok(newReservation);
}
```

How it works:

1. `spot.Type.Type == request.SpotType`: Narrows the search down to only the parking spots matching the string input (e.g., "Handicap", "Standard", "EV").
2. `!_context.ParkingReservations.Any(...)`: Checks the time overlap exactly like the previous endpoints to ensure the spot is truly free.
3. `.FirstOrDefaultAsync()`: Instead of fetching a list of all available spots, this immediately grabs the very first one that passes both filters. If all spots are taken (or the type doesn't exist), it returns null.

User prompt: using System.Security.Claims; using System.IdentityModel.Tokens.Jwt; using System.Text; using Microsoft.IdentityModel.Tokens; public interface ITokenService { string GenerateToken(string userId, string username, string role); ClaimsPrincipal? ValidateToken(string token); } public class TokenService : ITokenService { private readonly IConfiguration _configuration; public TokenService(IConfiguration configuration) { _configuration = configuration; } public string GenerateToken(string userId, string Email, string role) { var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"])); var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256); var claims = new[] { new Claim(JwtRegisteredClaimNames.Sub, userId), new Claim(JwtRegisteredClaimNames.Email, Email), new Claim(ClaimTypes.Role, role), new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()) }; var tokenDescriptor = new SecurityTokenDescriptor { Subject = new ClaimsIdentity(claims), Expires = DateTime.UtcNow.AddMinutes(double.Parse(_configuration["Jwt:ExpireMinutes"])), Issuer = _configuration["Jwt:Issuer"], Audience = _configuration["Jwt:Audience"], SigningCredentials = credentials }; var tokenHandler = new JwtSecurityTokenHandler(); var token = tokenHandler.CreateToken(tokenDescriptor); return tokenHandler.WriteToken(token); } public ClaimsPrincipal? ValidateToken(string token) { var tokenHandler = new JwtSecurityTokenHandler(); var key = Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]); try { tokenHandler.ValidateToken(token, new TokenValidationParameters { ValidateIssuerSigningKey = true, IssuerSigningKey = new SymmetricSecurityKey(key), ValidateIssuer = true, ValidIssuer = _configuration["Jwt:Issuer"], ValidateAudience = true, ValidAudience = _configuration["Jwt:Audience"], ClockSkew = TimeSpan.Zero }, out SecurityToken validatedToken); var jwtToken = (JwtSecurityToken)validatedToken; return new ClaimsPrincipal(new ClaimsIdentity(jwtToken.Claims, "jwt")); } catch { return null; } } } I want a method to get back userid

Response: Here are two ways you can get the User ID back, depending on where you need to access it.

1. Adding a method to your TokenService

If you have the raw token string and want a service method to extract the ID, you can add a `GetUserIdFromToken` method.

First, update your interface:

```
public interface ITokenService
{
    string GenerateToken(string userId, string username, string role);
    ClaimsPrincipal? ValidateToken(string token);
    string? GetUserIdFromToken(string token); // Add this line
}
```

Then, add the implementation to your `TokenService` class:

```
public string? GetUserIdFromToken(string token)
{
    // Validate the token to get the ClaimsPrincipal
    var principal = ValidateToken(token);

    if (principal == null)
    {
        return null; // Token was invalid or expired
    }

    // When ASP.NET Core creates the ClaimsPrincipal, it sometimes maps
    // "sub" to ClaimTypes.NameIdentifier. We check for both just to be safe.
    var userIdClaim = principal.FindFirst(ClaimTypes.NameIdentifier)
        ?? principal.FindFirst(JwtRegisteredClaimNames.Sub);

    return userIdClaim?.Value;
}
```

2. Getting it directly inside a Controller (Best Practice)

If you have properly configured JWT Authentication in your Program.cs, you **don't need to manually decode the token** in your controller. ASP.NET Core automatically parses the token from the Authorization header and populates the HttpContext.User property.

You can get the User ID directly in any Controller endpoint like this:

```
using System.Security.Claims;
using System.IdentityModel.Tokens.Jwt;

[Authorize] // Ensures the user is logged in
[HttpPost("reserve")]
public async Task<IActionResult> MakeReservation([FromBody] CreateReservationDTO request)
{
    // Extract the User ID from the HttpContext (the currently logged-in user)
    var userIdString = User.FindFirst(ClaimTypes.NameIdentifier)?.Value
        ?? User.FindFirst(JwtRegisteredClaimNames.Sub)?.Value;

    if (userIdString == null || !int.TryParse(userIdString, out int userId))
    {
        return Unauthorized("Invalid user token.");
    }

    // Now you can use `userId` for your database operations
    // e.g., user_id = userId

    // ... rest of your reservation logic ...
}
```

Why this is better for reservations: Extracting the userId directly from the authenticated User context is much more secure than passing UserId in the JSON body, as it prevents a malicious user from passing someone else's ID and making a reservation under their name.
